TRANSCRIPT

QA Evidence Transcript — LLMsVerifier verify-completion + honest scoring (§11.4.83)

Revision: 1 **Last modified:** 2026-06-17T00:00:00Z **Project:** helix_translate **Scope:** the verify-completion feature shipped this session — the LLMsVerifier now runs real per-model verification and persists honest 0.0–1.0 scores; helixtranslate exposes the verified set via `/api/v1/verified-models`. **Submodule commit:** `ed03a98c` (`llms_verifier/`) · **parent gitlink:** `9aa8d2d` **Discipline:** facts only (§11.4.6); every claim cites a source-evidence path. Unproven items are marked UNCONFIRMED. No secret values appear in this document (§11.4.10).

> NOTE — this is a CURATED record assembled from the session's raw QA evidence on T7. The raw evidence paths are cited verbatim so a future reader/operator can re-open the underlying artefacts. This transcript itself introduces no new claims beyond what those artefacts show.

---

0. One-line summary

The verifier verifies real models and persists honest scores (2 verified: `deepseek-chat` 0.80, `llama-3.3-70b-versatile` 1.0, reachable from the helixtranslate API in-container); an attempt to WIRE that verified set into the live translation path on nezha was **not viable**, was **auto-rolled-back**, and left translation **NON-degraded** on the working direct path (`llm-novita` / `llm-mistral`).

---

1. What the feature does + runtime signature

1.1 The feature

The LLMsVerifier submodule now (a) seeds candidate models from its `config.yaml` `llms:` at boot and (b) runs a real per-model verification pass (`POST /api/models/{id}/verify`) that makes live provider API calls and **persists an honest 0.0–1.0 score** plus a `verification_status`. Models that pass become `verification_status:"verified"`; helixtranslate reads the verified set through its `/api/v1/verified-models` reporting endpoint.

1.2 Runtime signature (FACT — count(verified)=2, two models + scores)

Captured live against the nezha deployment during the enable attempt (verifier reachable, the helixtranslate API serving the verified set):

- helixtranslate API `:18443 GET /api/v1/verified-models → {"count":2,"models":[ {"id":"deepseek-chat","name":"DeepSeek Provider","overall_score":0.7999999999999999,"verification_status":"verified"}, {"id":"llama-3.3-70b-versatile","name":"Groq Provider","overall_score":1,"verification_status":"verified"} ] }` — evidence: `helixtranslate_enable_20260616T222048Z/step2b_verified_models_after.json`.
- In-container, the verifier itself `http://llm-verifier:8080/api/models` returned the same 2 rows (`deepseek-chat` score `0.7999999999999999` `status:"verified"`, created `2026-06-16T21:32:09Z`) — evidence: `helixtranslate_enable_20260616T222048Z/step2a_post_recreate_state.txt`.

Note on endpoint naming: the **verifier-side** endpoint is `/api/models`; the **helixtranslate-side** reporting endpoint is `/api/v1/verified-models`. Both reported `count:2` with the same two models. (The task brief's shorthand "/api/models count(verified)=2" maps to these two captured surfaces.)

The numeric scores are honest persisted values: `0.7999999999999999` (deepseek-chat) and `1` (llama-3.3-70b-versatile / Groq) — not placeholders.

---

2. The verify-completion mechanism (handler → ModelVerificationService → persist) + auth-header fix

2.1 Verification path

- Verification is driven by the verifier's `VerifyModel` flow (`llms_verifier/llm-verifier/providers/model_verification_service.go`, `llms_verifier/llm-verifier/verification/code_verification.go`). It runs three real probes per model (`max_tokens:150, temperature:0.1`): a meaningful-response gate (`"hello!"`), a realistic-debate prompt, and a code-visibility probe over 5 fixed code samples. On a non-error result it persists `VerificationScore` and flips `verification_status` toward `"verified"`. — evidence: `research_verify_completion.md` §1 (quoted `file:line`).
- helixtranslate consumes the persisted verified set via its reporting handler (`pkg/api/verifier_handlers.go` `listVerifiedModels`, registered `~:100`), which calls the verifier client and returns `{"models":[...],"count":N}`; it is gated by `requireEnabled` (503 when the integration is disabled). — evidence: `llmsverifier_analysis.md` §Q3(a).
- The verified set only ENTERS translation through the bridge (`pkg/api/handler.go` `createTranslator → bridgeFor → pkg/bridge/bridge.go` `Open`); in HTTP mode the bridge's registry is fed from the verifier's `/api/models`. — evidence: `llmsverifier_analysis.md` §Q3(b), key file:line list.

2.2 Boot-seed (the prerequisite that makes verification have inputs)

- `seed.FromConfig(cfg, db)` is called in `runServer()` after DB init, **non-fatal**; `seed.go` `for i := range cfg.LLMs` idempotently upserts providers + models; seeded rows are written `VerificationStatus:"pending"` (NOT yet `"verified"`), so a real verification pass is required to reach `verified`. 6 real-in-memory-SQLite tests in `seed_test.go` (incl. a RED-baseline asserting `db.ListModels` returns the seeded rows + idempotency). — evidence: `llmsverifier_analysis.md` §Q1; committed `docs/qa/llmsverifier_wire_attempt_20260616T203122Z/STATE.md` §"INDEPENDENT review … GO".

2.3 The auth-header fix

When helixtranslate was first enabled against the verifier, the server **crash-looped** on config validation: *"LLMsVerifier API key is required when verifier is enabled"* — the env had an empty `LLMSVERIFIER_API_KEY`. The verifier's `/api/models` needs **no auth**, so the fix was to set a **NON-SECRET placeholder** key value purely to satisfy the "key is required when enabled" validation; no real secret was used or stored. After that, the server booted healthy and the verified-models endpoint returned the real 2-model set. — evidence: `helixtranslate_enable_20260616T222048Z/RESULT.md` STEP 1/2; `helixtranslate_enable_20260616T222048Z/CONFIG_TRACKING.md` (the appended env lines were non-secret).

---

3. Honest §11.4.150 finding — the score is a LIVENESS gate, not a quality signal

A deep multi-angle research pass (§11.4.150 / §11.4.99) evaluated whether the persisted 0.0–1.0 score is a sound basis for SELECTING a translation model. Finding (FACT, from code):

- The score is computed by **keyword counting** over a fixed "Do you see my code?" probe across 5 code samples — `+0.5` affirmative ("yes"/"I can see"), `+0.2` no-negative, `+0.1 × code-keyword-count`, plus an understanding-level bump — then averaged, with a **floor of 0.70** applied (`code_verification.go:153 max(ConfidenceScore, 0.7)`). It does **not** run a translation task and contains **zero translation-quality signal**.
- Consequence: the meaningful range is compressed to ~0.70–1.00, and the 1.0-vs-0.80 gap between groq/llama and deepseek-chat is **largely a phrasing/verbosity artifact**, not a quality finding. helixtranslate's gate (`MinScoreThreshold` default `0.0`) filters essentially nothing at default, and selection picks the highest score (descending sort).
- **Soundness verdict:** GOOD ENOUGH as a *liveness / "model reachable and answering coherently"* gate (real API calls, catches errors/refusals/timeouts, `verified+CanSeeCode+AffirmativeResponse` admission filter); **NOT SOUND** as a 0.0–1.0 quality rank for choosing a translation model.
- Cited external authorities establishing that single-prompt keyword scoring is below the bar for a capability rank: EleutherAI lm-

evaluation-harness; Stanford CRFM HELM (7 metrics × 42 scenarios).
- **UNCONFIRMED (honest gap):** whether the verify-completion `VerificationScore` actually becomes the registry `OverallScore` (the selection rank key), or only gates `verified` status while `OverallScore` is computed by the separate 5-component `scoring/engine.go`, was **not traced end-to-end** — flagged as a tracked follow-up.
- **Recommended follow-ups (NOT changes made this session):** treat the score as a binary gate, add translation-specific multi-sample scoring (chrF/COMET-class), raise `MinScoreThreshold` above 0.0 only after that, and close the score→OverallScore wiring UNCONFIRMED.

— evidence: `research_verify_completion.md` §1–§4 + Sources.

---

4. helixtranslate-enable outcome — NOT viable → auto-rolled-back → NON-degraded

An attempt to WIRE the verified set into the live translation path on nezha (`nezha.local`, deploy dir `/home/milosvasic/helixtranslate`) was executed and then reversed. Full bidirectional record:

Step	Action	Result	Evidence
0 — baseline	`:18443 POST translate` "Good morning" en→sr	`{"translated":"Добро јутро","provider":"llm-mistral","Errors":0}` WORKING	`helixtranslate_er step0_baseline_t`
1 — enable	added `LLMSVERIFIER_ENABLED=true` + `LLMSVERIFIER_API_URL=http://llm-verifier:8080` to nezha-local `.env.nezha`; connected 4 app containers to `llmsverifier_default` net	first boot crash-looped on empty `LLMSVERIFIER_API_KEY` → fixed with non-secret placeholder (§2.3)	`helixtranslate_er` 1/2; `.../step1_rec step1b_recreate_`
2 — post-enable	verifier reachable + verified-models served	`verified-models count:2` SUCCESS, but `:18443 POST translate` → ERROR *"bridge: no verified translator available: no verified*	`.../step2b_verif step2c/2d_transl step2_FAIL_trans`

Step	Action	Result	Evidence
		models available meeting threshold 0.00" (HTTP exit 56) — DEGRADATION	
3 — auto-rollback	restored original `.env.nezha`, disconnected app containers from `llmsverifier_default`, recreated containers	baseline restored	`.../step3_rollbac` `step3a_post_roll`
3b/3c — confirm	re-ran translation in final state	`{"translated":"Добро утро","provider":"llm-novita","Errors":0}` (exit 0); 2nd phrase translates, 0 errors; verified-models → "integration disabled"; all 6 containers healthy	`.../` `step3b_ROLLBACK_` `step3c_rollback_`

Root cause of the degradation (FACT)

Enabling the verifier **reroutes** translation through the bridge, which selects only VERIFIED models (`deepseek-chat` / `llama-3.3-70b-versatile`). Those verified providers had **no usable translator binding** in the nezha deployment (no provider keys wired for them on the translate path), so the bridge returned *"no verified translator available"* and translation FAILED — while the working direct multi-provider path (`mistral`/`novita`) was bypassed. In HTTP-bridge mode there is **no graceful fallback** to the direct path (`bridge.Open` returns in the HTTP branch before in-process is reached). The never-degrade precondition was violated → **AUTO-ROLLBACK**. — evidence: `helixtranslate_enable_.../RESULT.md` "Root cause of degradation"; `llmsverifier_analysis.md` §Q3(b).

Final state (NON-degraded — §11.4.69 never-degrade proof)

Translation works on the direct path (`llm-novita`, "Добро утро", 0 errors), verified-models endpoint cleanly reports disabled, all 6 containers healthy. Net config change to the deployment is ZERO (the nezha-local `.env.nezha` is gitignored secrets and was restored to its original; nothing to commit there). The earlier honest-blocked verdict (`docs/qa/llmsverifier_wire_attempt_20260616T203122Z/STATE.md`,

20:31Z) and this later enable-attempt-then-rollback (22:20Z → 01:37) agree on the outcome: **LLMsVerifier remains a quality enhancement, not a translation dependency; the direct path stays intact.**

---

5. Cited evidence paths (raw artefacts on T7)

Raw evidence (read-only source material for this transcript):

- `/Volumes/T7/helix-build/qa/llmsverifier_analysis.md` — seed-at-boot + bridge HTTP-vs-in-process safety analysis (code-quoted, `file:line`).
- `/Volumes/T7/helix-build/qa/research_verify_completion.md` — §11.4.150 deep-research on the verify-completion scoring methodology (liveness-gate-not-quality finding + cited sources).
- `/Volumes/T7/helix-build/qa/helixtranslate_enable_20260616T222048Z/` — the enable→fail→rollback run: `RESULT.md`, `CONFIG_TRACKING.md`, `step0`…`step3c` captured outputs.

Committed-in-repo evidence (durable record):

- `/Volumes/T7/Projects/helix_translate/docs/qa/llmsverifier_wire_attempt_20260616T203122Z/STATE.md` (+ `.html`/`.pdf`) — honest-blocked verdict + §11.4.142 independent GO review of the seed code.
- `/Volumes/T7/Projects/helix_translate/docs/qa/llmsverifier_wire_attempt_20260616T203122Z/INDEPENDENT_ANALYSIS_6e6b0309.md` — independent analysis of the seed submodule commit.

Code references (helix_translate working tree, read-only):

- `llms_verifier/llm-verifier/providers/model_verification_service.go`, `.../verification/code_verification.go` — VerifyModel + scoring.
- `llms_verifier/llm-verifier/seed/seed.go`, `.../seed/seed_test.go`, `.../cmd/main.go`, `.../config.yaml` — boot-seed.
- `pkg/api/verifier_handlers.go` — `/api/v1/verified-models` reporting handler.
- `pkg/api/handler.go` `createTranslator`/`bridgeFor`; `pkg/bridge/bridge.go` `Open`/`BestModel` — the translation bridge (HTTP vs in-process).

- `internal/verifier/scoring/engine.go`, `internal/verifier/selection/engine.go`, `internal/verifier/registry.go`, `internal/verifier/config.go` — score/select/gate.

---

6. Auditable conclusions (anti-bluff)

1. The verify-completion feature **really runs** and **persists honest scores** — proven by the live 2-verified-model runtime signature captured in-container and at the helixtranslate API (§1.2).
2. The scores are a **liveness gate, not a translation-quality rank** — an honestly-recorded limitation with cited external authorities, plus one UNCONFIRMED wiring gap left tracked (§3).
3. Wiring the verified set into live translation is **not viable in the current nezha deployment** (verified providers lack translator bindings), was **auto-rolled-back**, and translation is **confirmed working NON-degraded** on the direct path (§4).
4. Net deployment change = ZERO; no secret values appear in any artefact or in this transcript (§11.4.10).